

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – CASE STUDY

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO3: Articulation (BL4, BL5)	Assessment:	Assessment Tool 2 – Case Study
Weightage:	20% of CIA	Total Marks:	100
CO3 Statement:	Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication		
Student Name:	K NAVYA	Reg. No.:	192371022
Section / Batch:	CSE BIO	Date:	06/08/2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given case study questions.
- Explain in detail with sample code if needed.
- Clearly identify all key issues, provide an in-depth analysis, and propose innovative, feasible solutions with strong justification.

Question Type	Focus Area	Questions
Case Study	Focus on Design Divide and Conquer algorithms: Merge Sort, Quick Sort, Binary Search and Matrix Multiplication	Q1

SECTION A – CASE STUDY

Code of Conduct

I K NAVYA (192371022) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Case Study					
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25%	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25%	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20%	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20%	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10%	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25%				
Application of Concepts	25%				
Analysis	20%				
Solution Design	20%				
Clarity	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name: _____	Name: _____	Name: _____
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q1. Case Study: E-Commerce Product Sorting System (Merge Sort)

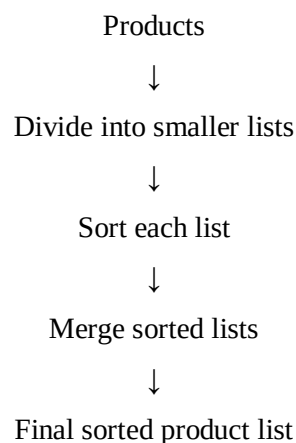
An e-commerce platform must sort millions of products based on price and ratings to ensure fast user access. Analyze how Merge Sort can be used to handle large datasets efficiently.

Clearly identify key issues such as stability and scalability. Apply divide and conquer concepts to explain the sorting process. Analyze its time complexity and performance. Design a feasible solution and justify why Merge Sort is suitable for this scenario.

Merge Sort works in three steps:

1. Divide: The product list is divided into two smaller halves.
2. Conquer: Each half is recursively sorted.
3. Merge: The sorted halves are combined to form one completely sorted list.

For example:



Key Issues

1. Stability:

Merge Sort is a stable sorting algorithm. If two products have the same price, their original order can be preserved. This is useful when sorting products using multiple criteria, such as price first and rating second.

2. Scalability:

Merge Sort performs well even when the number of products becomes very large. It is therefore suitable for millions of product records.

3. Memory Usage:

Merge Sort generally requires $O(n)$ extra memory for the merging process. For extremely large datasets, External Merge Sort can be used to sort data stored on disk when it cannot fit into main memory.

Time Complexity

Case	Time Complexity
------	-----------------

Best Case	$O(n \log n)$
-----------	---------------

Average Case	$O(n \log n)$
--------------	---------------

Worst Case	$O(n \log n)$
------------	---------------

Its predictable $O(n \log n)$ performance makes it more reliable for large datasets than simple algorithms such as Bubble Sort, which can take $O(n^2)$ time.

Feasible Solution

The e-commerce system can divide millions of product records into smaller groups, sort each group using Merge Sort, and then merge the sorted groups. Products can be sorted using a comparison rule such as:

Primary: Price → Secondary: Rating

For very large datasets, sorted chunks can be stored on disk and merged later using External Merge Sort.

Conclusion

Merge Sort is a suitable choice for an e-commerce product sorting system because it provides $O(n \log n)$ time complexity, stability, scalability, and reliable performance. Its Divide and Conquer approach makes it effective for sorting millions of products efficiently.

Q2. Case Study: Real-Time Ranking System (Quick Sort)

A real-time ranking system updates scores dynamically and needs fast sorting for leaderboard display. Analyze how Quick Sort can be applied in this scenario. Identify issues such as pivot selection and worst-case performance. Apply divide and conquer concepts to explain its working. Analyze its efficiency under different cases. Propose an optimized solution and justify improvements like randomized pivot selection.

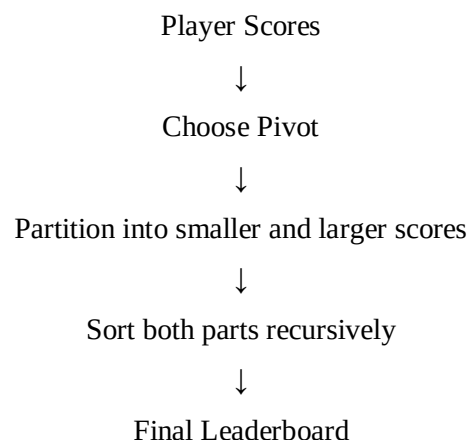
A real-time ranking system such as a **leaderboard** needs to frequently sort players based on their scores. Since scores can change dynamically, the sorting algorithm must be fast and efficient. **Quick Sort** is suitable because it is generally very fast and works efficiently in memory.

Divide and Conquer

Quick Sort follows three steps:

1. **Divide:** Select an element as a **pivot** and partition the list into elements smaller and larger than the pivot.
2. **Conquer:** Recursively apply Quick Sort to the two partitions.
3. **Combine:** The sorted partitions together with the pivot form the final sorted list.

For example:



Key Issues

1. Pivot Selection:

The choice of pivot greatly affects performance. A poor pivot, such as repeatedly choosing the smallest or largest element, can create highly unbalanced partitions.

2. Worst-Case Performance:

If partitions are very unbalanced, Quick Sort can take $O(n^2)$ time. This may happen when the data is already sorted and a bad pivot is consistently selected.

3. Dynamic Updates:

When scores change, the leaderboard may need to be sorted again. Efficient pivot selection helps reduce the sorting time.

Time Complexity

Case	Time Complexity
------	-----------------

Best Case	$O(n \log n)$
-----------	---------------

Average Case	$O(n \log n)$
--------------	---------------

Worst Case	$O(n^2)$
------------	----------

Quick Sort generally requires $O(\log n)$ auxiliary stack space for balanced recursion and is usually performed **in-place**, so it needs less extra memory than Merge Sort.

Optimized Solution

A good solution is to use **Randomized Quick Sort**, where the pivot is selected randomly instead of always choosing a fixed position. This reduces the chance of repeatedly getting poor partitions.

Other improvements include:

- Use **randomized pivot selection**.
- Use the **median-of-three** method for better pivot selection.
- Use **Insertion Sort for very small partitions**.
- Avoid unnecessary sorting when only a small leaderboard section needs updating.

Conclusion

Quick Sort is a good choice for a real-time ranking system because it provides **fast average-case performance, low extra memory usage, and efficient in-place sorting**. Using a **randomized pivot** significantly reduces the risk of worst-case behaviour and makes the algorithm more reliable for dynamic leaderboards.

Q3. Case Study: Search Engine Query Processing (Binary Search)

A search engine maintains a sorted index of keywords and must retrieve results quickly.

Analyse how Binary Search improves search efficiency. Identify key requirements such as sorted data and fast retrieval. Apply divide and conquer principles to explain the process.

Analyse time complexity and performance. Design a solution and justify why Binary Search is optimal in this context.

A search engine maintains a large **sorted index of keywords**. When a user searches for a keyword, the system must find it quickly and retrieve the related results. **Binary Search** is highly suitable because it reduces the search space by half at every step.

Key Requirements

- The keyword index must be **sorted**.
- Search results should be retrieved **quickly**.

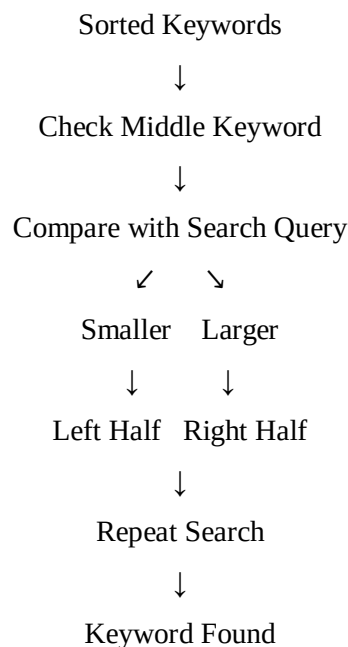
- The system should handle a **large number of keywords** efficiently.
- The search process should use minimum comparisons.

Divide and Conquer

Binary Search follows the Divide and Conquer approach:

1. **Divide:** Find the middle element of the sorted list.
2. **Conquer:** Compare the search keyword with the middle element.
3. If the keyword is smaller, search the **left half**.
4. If it is larger, search the **right half**.
5. Repeat until the keyword is found or no elements remain.

For example:



Time Complexity

Case	Time Complexity
Best Case	$O(1)$
Average Case	$O(\log n)$
Worst Case	$O(\log n)$

Binary Search requires **$O(1)$** extra space when implemented iteratively.

For example, if there are **1,000,000 keywords**, Binary Search needs only around **20 comparisons** in the worst case because the search space is repeatedly divided by two.

Proposed Solution

The search engine can maintain its keyword index in sorted order. Whenever a user enters a query, Binary Search can quickly locate the keyword and return its associated search results.

If the keyword index changes frequently, it must be kept sorted or periodically rebuilt before Binary Search is applied.

Why Binary Search Is Optimal

Binary Search is an excellent choice because it:

- Provides **$O(\log n)$** search time.

- Quickly handles very large sorted datasets.
- Reduces the search area by half at every step.
- Requires very little extra memory.
- Provides much faster searching than **Linear Search ($O(n)$)**.

Conclusion

Binary Search is highly effective for search engine query processing when keywords are stored in a **sorted index**. Its Divide and Conquer approach and **$O(\log n)$** time complexity make it a fast and memory-efficient solution for retrieving keywords from large datasets.

Q4. Case Study: Image Processing System (Matrix Multiplication)

An image processing system performs transformations using large matrices. Analyse how divide and conquer matrix multiplication (e.g., Strassen's method) can improve performance. Identify computational challenges in large matrix operations. Apply the divide and conquer approach to explain optimization. Analyse time complexity and compare with standard multiplication. Design a suitable solution and justify its efficiency.

An image processing system often uses large matrices to perform operations such as image transformation, scaling, rotation, and filtering. Multiplying large matrices using the standard method can be computationally expensive. Divide and Conquer matrix multiplication, especially Strassen's method, can improve performance for large matrices.

Computational Challenges

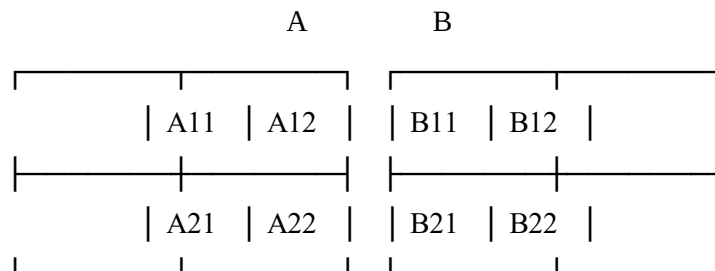
Large matrix operations involve:

- A large number of multiplication and addition operations.
- High processing time for large images.
- Increased memory usage.
- Performance problems when matrices become very large.

Divide and Conquer Approach

In standard matrix multiplication, multiplying two $n \times n$ matrices takes **$O(n^3)$** time.

Strassen's method divides each matrix into four smaller submatrices:



Divide into smaller matrices



Perform recursive operations



Combine the results

Instead of performing **8 recursive multiplications**, Strassen's algorithm uses only **7 multiplications** along with additional additions and subtractions. This reduces the total computational work for sufficiently large matrices.

Time Complexity

Method	Time Complexity
Standard Matrix Multiplication	$O(n^3)$
Strassen's Method	Approximately $O(n^{2.807})$

Thus, Strassen's method is faster for sufficiently large matrices.

Suitable Solution

For an image processing system, Strassen's algorithm can be used when the matrices are large. The matrices are divided into smaller blocks, processed recursively, and then combined to obtain the final result.

For small matrices, standard multiplication can be used because Strassen's extra additions and recursion may create unnecessary overhead.

Advantages

- Reduces the number of multiplication operations.
- Faster for large matrices.
- Uses Divide and Conquer effectively.
- Suitable for large-scale image processing and transformations.

Conclusion

Strassen's Matrix Multiplication is a suitable solution for processing large image matrices. Its $O(n^{2.807})$ time complexity is better than the standard $O(n^3)$ approach. A **hybrid solution**, using Strassen's method for large matrices and standard multiplication for small matrices, provides better overall performance and efficiency.

Q5. Case Study: Data Analytics Platform (Merge vs Quick Sort)

A data analytics platform must process large datasets efficiently for reporting. Analyse the use of Merge Sort and Quick Sort in this scenario. Identify key issues such as memory usage, speed, and stability. Apply divide and conquer concepts to compare both algorithms. Analyse their performance under different conditions. Design a solution selecting the best algorithm and justify your choice.

A data analytics platform processes large datasets for generating reports. Sorting is important for arranging data based on values such as sales, dates, prices, or customer records. Both Merge Sort and Quick Sort use the Divide and Conquer approach, but they differ in memory usage, speed, and stability.

Divide and Conquer Approach

Merge Sort:

- Divides the dataset into two halves.
- Recursively sorts both halves.
- Merges the sorted halves.
- Requires extra memory for merging.

Quick Sort:

- Selects a pivot element.
- Divides the data into smaller and larger elements.
- Recursively sorts both partitions.
- Usually works in-place and needs less extra memory.

Comparison

Feature	Merge Sort	Quick Sort
Best Case	$O(n \log n)$	$O(n \log n)$
Average Case	$O(n \log n)$	$O(n \log n)$
Worst Case	$O(n \log n)$	$O(n^2)$
Extra Memory	$O(n)$	$O(\log n)$ average
Stability	Stable	Usually not stable
Speed	Consistent	Usually faster in practice

Performance Under Different Conditions

Merge Sort provides consistent performance even when the data is already sorted or arranged in an unfavorable order. It is also useful when stability is important.

Quick Sort is often faster in practice because it works in-place and has good cache performance. However, poor pivot selection can cause $O(n^2)$ performance. Using a randomized pivot or median-of-three selection can reduce this risk.

Proposed Solution

For a data analytics platform, Merge Sort is the better overall choice when datasets are very large and reports require stable and predictable sorting. It can also be adapted for external sorting when data does not fit into memory.

Quick Sort can be preferred when memory is limited and fast in-memory sorting is required, especially with randomized pivot selection.

Conclusion

Both algorithms are efficient Divide and Conquer techniques. Quick Sort generally provides faster in-memory performance and uses less extra memory, while Merge Sort offers guaranteed $O(n \log n)$ performance and stability. For large analytics datasets where reliability and stable results are important, Merge Sort is the recommended choice.